



PART 11 · 30 MIN · CLOSING

Q&A, Exam Briefing & Next Steps

Ten projects done. Now: the three frameworks you keep, the exam, and Monday.

Theory · Three frameworks you keep (4 min)

Forget the syntax; keep these three:

- 1 **The loop** — Plan → Implement → Test → Review → Commit (every non-trivial change). *Module 1.*
- 2 **The 40/40/20 rubric** — grade any AI output: 40% correctness · 40% quality · 20% fit. *Module 5 + assessment.*
- 3 **The readiness checklist** — five axes before you tag a release. *Module 10.*

If you remember nothing else, remember the loop. It is the whole course in five words.

Reference · The five most common mistakes

1. **No plan** — jumping straight to "write the function".
2. **Skipping review** — accepting the first diff.
3. **Letting Claude commit** — losing the human checkpoint.
4. **Treating skills as files** instead of habits invoked deliberately.
5. **No CLAUDE.md** — re-explaining stack + conventions every session.

Every one is a habit, not a knowledge gap. Fix the habit.

Reference · Three prompting anti-patterns

Anti-pattern	Symptom	Fix
"Fix it" loop	Vague prompt → unfocused diff → re-prompt → drift	Paste the exact error + smallest reproducer
Over-eager agent	Long run → wrong abstraction → 600-line diff	Stop at the plan, review, <i>then</i> implement
Merge-without-review	Claude commits + pushes in one shot	Review-before-commit, even when "obviously fine"

Live demo · "Fix it" loop vs. precise prompt (4 min)

1. Reproduce the **"fix it" loop** — vague prompt → unfocused diff → drift:

```
It's broken, fix it.
```

2. Reset. Paste the precise prompt — exact error + smallest reproducer:

```
GET /notes/999 returns 500, expected 404. KeyError 'note' in get_note() line 42.  
Fix only this; keep all other behavior. Show the diff.
```

3. Narrate: the difference between coaching and guessing.

Success signal: the precise prompt fixes it in one pass; the vague loop doesn't.

Exam briefing · How to pass

Submit (zip to the Packt LMS):

- The three assessment artefacts from `assessments/`: knowledge-quiz · practical-task · code-review-reflection.
- Weighted score $\geq 70\%$ against `assessments/rubric.md`.

Be ready to:

- Name the five May 2026 pillars from memory: **Skills · Hooks · MCP · GitHub Actions · Multi-agent.**
- Answer in one sentence: *"What will I try in my own repo on Monday?"*

Future-proof · Keep an eye on (May 2026 → beyond)

The tools change monthly; the **habits** don't. What to watch — and the trick that compounds:

Watch in 2026	Trick that makes life better
Shared skill libraries across teams	Keep a <code>skills/</code> folder in every repo — borrow habits, don't reinvent
MCP servers for more of your stack	Wire issue tracker · CI · observability in once; verify with <code>/mcp</code>
Multi-agent orchestration maturing	Delegate parallel work, keep one human reviewer — you
Hooks as default guardrails	Auto-format, secret-scan, test-gate on every action
Bigger context + memory files	Pin model + conventions in <code>CLAUDE.md</code> ; it still wins

The rule that survives every release: **Plan → Implement → Test → Review → Commit.**

▶ Your turn · The Monday sentence (3 min)

No code this time — one sentence. Complete it and write it where you'll see it:

"On Monday, I will use Claude Code to ____ on my project ____, and I will stop the loop when ____."

Then:

- Upload your submission zip to the Packt LMS.
- ★ Star the repo so you can find the skills on Monday.

Success signal: you can say your Monday sentence out loud without hesitating.

✓ **Done · You're certified-ready (1 min)**

Definition of done — the whole bootcamp

- ☐ [] Three assessment artefacts in the submission zip.
- ☐ [] Weighted score $\geq 70\%$ against the rubric.
- ☐ [] Can name the five May 2026 pillars from memory.
- ☐ [] Have your one-sentence Monday answer.

Thank you. You direct, you review, you merge — you're the engineer of record. Go ship.